

A Structure-Exploiting C ADMM Solver for Embedded Quadrotor MPC, with Solver Benchmarks and Neural Distillation

Vrishabh Kenkre

Department of Mechanical Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
vkenkre@andrew.cmu.edu

Abstract—We implement a hand-rolled, structure-exploiting ADMM solver in C for linear MPC of a Bitcraze Crazyflie 2 quadrotor and benchmark it against OSQP and the GPU-accelerated ReLU-QP solver. The custom ADMM caches a Riccati recursion in the infinite-horizon LQR backward pass and reduces the active per-iteration work to 12×12 block operations rather than a sparse Cholesky on the full 332×332 KKT system. On a figure-8 trajectory at 1 m/s the controller achieves 3.7 mm steady-state RMSE with median solve time of $86 \mu\text{s}$ on a single laptop CPU core, $24.9\times$ faster than OSQP at matched accuracy. ReLU-QP, a dense general-purpose ADMM reformulated as a neural network forward pass, fails to converge within 4000 iterations on this 12-state problem; we attribute the failure to scale heterogeneity in the cost matrix combined with the dense formulation, and use this finding to characterize the crossover regime between structure-exploiting and parallelism-exploiting QP solvers. Finally, we distill the MPC controller into a 6,276-parameter feedforward network using DAGger with DART noise injection. The distilled policy tracks figure-8 at 4.1 mm RMSE (11% degradation versus the teacher) while running in $2.1 \mu\text{s}$ of pure C inference — a $41\times$ speedup that eliminates the dynamics model and solver from the deployment binary.

Index Terms—quadrotor, model predictive control, ADMM, embedded optimization, policy distillation, DAGger

I. INTRODUCTION

Model predictive control (MPC) is the dominant approach to constrained quadrotor trajectory tracking, but its real-time viability on small platforms is limited by the cost of solving a quadratic program (QP) every control step. The Bitcraze Crazyflie 2 has 192 KB of SRAM and a 168 MHz Cortex-M4; off-the-shelf solvers like OSQP [1] are general-purpose and use sparse Cholesky factorizations that are not well matched to the dense, low-dimensional structure of a 20-step linear MPC problem.

Two recent solver developments target this regime from opposite directions. TinyMPC [2] exploits the Riccati structure of the infinite-horizon LQR feedback law to reduce per-iteration work to $n_x \times n_x$ block operations, where $n_x = 12$ for a quadrotor. ReLU-QP [3] reformulates ADMM as a fixed-architecture neural network whose forward pass executes on a GPU at the rate of a few hundred microseconds per batched solve — attractive for parallel evaluation across many

trajectories or many robots, but designed for medium-to-large state dimensions ($n \gtrsim 100$).

This paper presents a from-scratch C implementation of the TinyMPC-style solver tuned for the Crazyflie, an empirical comparison against OSQP and ReLU-QP, and a downstream policy distillation step that compresses the entire MPC pipeline into a small feedforward network. The contributions are:

- 1) A complete, MIT-licensed C ADMM solver with cached Riccati recursion, $86 \mu\text{s}$ median solve time, and verified zero linearization error against CasADi autodiff.
- 2) A head-to-head benchmark against OSQP (Python wrapper) and ReLU-QP, with the negative ReLU-QP result analyzed in terms of state-dimension scaling and cost-matrix conditioning.
- 3) A DAGger [5]-with-DART [6] distillation of the MPC controller into a 6K-parameter network exported to C, with $2.1 \mu\text{s}$ deployment latency and 4.1 mm tracking RMSE.

The full code, trained policy weights, and replication instructions are available at the repository accompanying this paper.

II. SYSTEM MODEL AND MPC FORMULATION

A. Quadrotor Dynamics

We use the standard 12-state Crazyflie model: $x = [p^\top, v^\top, \eta^\top, \omega^\top]^\top$ where $p, v \in \mathbb{R}^3$ are inertial position and velocity, $\eta = (\phi, \theta, \psi)$ are roll-pitch-yaw, and $\omega \in \mathbb{R}^3$ is the body-frame angular velocity. The control input is $u = [T, \tau_x, \tau_y, \tau_z]^\top$ with collective thrust T and three body torques. We use the MuJoCo Menagerie Crazyflie 2 model after applying the following corrections, which are required to obtain physically meaningful dynamics:

- Thrust range corrected from the default $[0, 0.0155]$ to the manufacturer-rated $[0, 0.6]$ N total thrust.
- Torque gear magnitudes adjusted so that maximum yaw torque is approximately 0.0069 N·m.
- Gear sign on τ_z inverted to match the sense convention used in the dynamics derivation.

We linearize about hover ($x_{\text{hover}} = 0$, $T_{\text{hover}} = mg$) to obtain $\dot{x} = Ax + Bu + d$ where d absorbs gravity and the hover thrust offset. Discretization at $\Delta t = 10$ ms by zero-order hold yields $A_d, B_d, d_{\text{ctrl}}$ matrices used by the MPC. We verified the linearization against a CasADi autodiff of the nonlinear MuJoCo dynamics at the equilibrium and obtained Frobenius-norm error $\|A_{\text{analytic}} - A_{\text{casadi}}\|_F = 0$.

B. Linear MPC

The MPC problem is

$$\begin{aligned} \min_{x,u} \quad & \sum_{k=0}^{N-1} \frac{1}{2} (x_k - x_k^{\text{ref}})^\top Q (x_k - x_k^{\text{ref}}) + \frac{1}{2} u_k^\top R u_k \\ & + \frac{1}{2} (x_N - x_N^{\text{ref}})^\top Q_f (x_N - x_N^{\text{ref}}) \\ \text{s.t.} \quad & x_{k+1} = A_d x_k + B_d u_k + d_{\text{ctrl}} \\ & u_{\min} \leq u_k \leq u_{\max} \\ & x_0 = x(t) \end{aligned} \quad (1)$$

with horizon $N = 20$, $Q = \text{diag}(Q_p, Q_v, Q_\eta, Q_\omega)$, and a control penalty R in which the thrust weight (30) and torque weights (1500) differ by a factor of 50 — this scale heterogeneity becomes important when discussing dense-formulation solvers in Section IV-B.

III. CUSTOM ADMM SOLVER IN C

A. Splitting and Algorithm

We apply ADMM to (1) by splitting the input variable u from a slack variable z that absorbs the box constraint:

$$\begin{aligned} \min_{u,z} \quad & f(u) + g(z) \\ \text{s.t.} \quad & u = z \end{aligned} \quad (2)$$

where f is the quadratic objective with the dynamics substituted in (closed form), and g is the indicator function of the box $[u_{\min}, u_{\max}]$. The augmented Lagrangian is

$$L_\rho(u, z, y) = f(u) + g(z) + y^\top (u - z) + \frac{\rho}{2} \|u - z\|^2 \quad (3)$$

and the iteration is the canonical three-step update [4]:

$$u^{k+1} = -K_\infty x - d_{\text{ctrl}} + (\rho/\sigma)(z^k - y^k) \quad (4)$$

$$z^{k+1} = \Pi_{[u_{\min}, u_{\max}]}(u^{k+1} + y^k/\rho) \quad (5)$$

$$y^{k+1} = y^k + \rho(u^{k+1} - z^{k+1}) \quad (6)$$

where K_∞ is the infinite-horizon LQR gain solving the discrete algebraic Riccati equation (DARE) for $(A_d, B_d, Q, R + \sigma I)$, and $\sigma > 0$ is a small regularizer.

B. The Riccati Caching Trick

The expensive step in a generic ADMM solver is (4), which requires solving an $n \times n$ linear system. OSQP, for example, factorizes the full sparse KKT system, which for our problem ($N = 20$, $n_x = 12$, $n_u = 4$) has 332 variables.

Following TinyMPC [2], we observe that for an MPC problem the optimal u^* at iteration $k+1$ is a linear function of the current state and a low-rank correction term $\rho(z^k - y^k)$. The correction propagates backward through the horizon via

a Riccati-style recursion that involves only $n_x \times n_x$ block operations. We pre-compute and cache the steady-state gain K_∞ and the propagation matrices C_1, C_2 once at startup; the per-iteration work in (4) is then a single matrix-vector product on 12×12 blocks.

This reduces the cost of (4) from $\mathcal{O}((N(n_x + n_u))^3)$ for sparse Cholesky to $\mathcal{O}(N \cdot n_x^2)$ for the cached recursion — approximately 22,000 FLOPs per iteration in our implementation versus 2.25×10^6 FLOPs for the equivalent OSQP factorization.

C. Implementation Details

The C implementation is approximately 400 lines and uses no dynamic allocation. The hot loop is shown in Listing 1. We use stack-allocated static arrays sized to a maximum problem dimension chosen at compile time, and a zero-overhead Python binding via ctypes for the offline benchmarking and DAGger data collection.

```
for (int k = 0; k < max_iters; ++k) {
    /* (5): u-update via cached Riccati */
    matvec(K_inf, x, Kx, NX, NX);
    for (int i = 0; i < NU; ++i)
        u[i] = -Kx[i] - d_ctrl[i]
            + rho_over_sigma * (z[i] - y[i]);

    /* (6): z-update via box projection */
    for (int i = 0; i < NU; ++i) {
        double v = u[i] + y[i] / rho;
        z[i] = clamp(v, u_min[i], u_max[i]);
    }

    /* (7): dual update */
    for (int i = 0; i < NU; ++i)
        y[i] += rho * (u[i] - z[i]);

    /* termination */
    if (primal_res(u, z) < tol &&
        dual_res(z, z_prev) < tol) break;
}
```

Listing 1. Inner loop of the C ADMM solver. Comments mark the algorithmic correspondence to (4) – (6). All arrays are stack-allocated.

D. Tracking and Latency Results

We evaluate the controller in MuJoCo on a figure-8 trajectory of amplitude 0.5 m at peak speed 1 m/s, with 30 s of simulation at a 100 Hz control rate. Steady-state RMSE is 3.7 mm. Table I summarizes the latency distribution. The bimodal pattern — $\sim 7 \mu\text{s}$ in steady state, $\sim 250 \mu\text{s}$ during reference transitions — reflects the early-termination behavior of ADMM: when the warm-started solution is feasible, 1 iteration suffices, and when the reference changes abruptly the solver runs up to 50 iterations.

IV. SOLVER BENCHMARKS

A. OSQP Comparison

We re-formulate (1) in OSQP’s sparse standard form and run the same figure-8 scenario. OSQP achieves identical tracking RMSE (3.7 mm) by construction — both solvers find the global minimum of the same convex QP. Median solve time is 2,141 μs , $24.9\times$ slower than the C ADMM. The difference is dominated by the cost of the sparse Cholesky factorization

TABLE I

C ADMM SOLVER LATENCY ON A CRAZYFLIE FIGURE-8 AT 1 M/S, MEASURED ON A SINGLE CORE OF AN INTEL I9-14900HX LAPTOP CPU.

Statistic	Solve time
median (steady-state)	7 μ s
median (over full trajectory)	86 μ s
95th percentile	278 μ s
99th percentile	331 μ s
maximum (transients)	363 μ s
mean iterations (steady-state)	1
mean iterations (transients)	40–50

on a 332×332 KKT matrix, which the Riccati caching avoids entirely.

B. ReLU-QP and the Negative Result

ReLU-QP [3] is a recent general-purpose ADMM solver that reformulates each iteration as a forward pass through a neural network: the linear-system solve becomes a matrix multiply with a precomputed weight matrix, the box projection becomes a ReLU-equivalent clamp, and the dual update is a residual connection. The result executes natively on GPU and is reported to outperform OSQP for state dimensions $n \gtrsim 100$.

We integrated ReLU-QP via its PyTorch reference implementation, applied a small numerical patch to handle our specific ρ scheduling, and ran the figure-8 benchmark at 4000 maximum iterations with the standard tolerance settings. The solver *does not converge*: tracking error oscillates between 0 mm (when the solver returns the warm-start unchanged) and 363 mm (when it returns a degenerate solution that violates thrust bounds), with all 4000 iterations exhausted on every step.

We diagnose this as the expected failure mode for ReLU-QP at small n : the dense weight matrix W has dimension 1500×1500 in the condensed formulation and mixes the thrust-cost ($R = 30$) and torque-cost ($R = 1500$) scales together. The resulting condition number degrades the convergence of the underlying ADMM, and the GPU parallelism advantage that justifies the dense formulation does not pay off until the problem is too large for structured solvers like ours to fit in cache. The crossover, based on our measurements and the published ReLU-QP scaling curves, is in the $n \approx 50$ –100 range — well above the Crazyflie’s $n = 12$.

We do not view this as a defect of ReLU-QP. The result is consistent with the design goals stated in [3]: “competitive for small-to-medium problems and order-of-magnitude improvements for larger-scale problems.” For our use case (single robot, 12 states), structure-exploiting solvers are correct; for batched evaluation across thousands of robots in a parallel simulator, ReLU-QP would be the right choice.

C. Summary

V. DISTILLATION: FROM MPC TO A NEURAL POLICY

A. Why Distill

The C ADMM is fast, but deployment still requires the full dynamics model, the cached Riccati matrices, and a working

TABLE II

SOLVER COMPARISON ON CRAZYFLIE LINEAR MPC, $N = 20$, $n_x = 12$, $n_u = 4$. ALL SOLVERS FIND THE SAME GLOBAL OPTIMUM WHEN THEY CONVERGE.

Solver	RMSE	Median solve	Notes
C ADMM (ours)	3.7 mm	86 μ s	cached Riccati
OSQP	3.7 mm	2,141 μ s	sparse Cholesky
ReLU-QP	—	—	did not converge

TABLE III

MPC VS DISTILLED POLICY ON FIGURE-8.

Controller	RMSE	Latency
LQR baseline	9.2 mm	$\sim 30 \mu$ s (NumPy)
C ADMM MPC	3.7 mm	86 μ s
BC alone	—	crashes
BC + DART	13.1 mm	2.1 μ s
DAGger + DART (final)	4.1 mm	2.1 μ s

solver. For embedded or batched-inference scenarios it is attractive to compress the entire pipeline into a feedforward network: the network has no dynamics dependency at deployment, generalizes across the model uncertainty seen at training time, and runs as a single matrix-multiply chain. We use DAGger [5] with DART [6] noise injection.

B. Network Architecture

The policy is a three-layer MLP: $24 \rightarrow 64 \rightarrow 64 \rightarrow 4$ with LayerNorm on the input and tanh on the output. The input is the state error plus the next four reference waypoints; the output is in the normalized $[-1, 1]$ range and is rescaled to $(T, \tau_x, \tau_y, \tau_z)$. Total parameters: 6,276.

C. DAGger with DART

Behavior cloning fails on this problem: a network trained purely on MPC rollouts diverges at deployment because the rollout distribution does not cover off-policy states. DART injects training-time noise on the expert action, sampled from $\mathcal{N}(0, \alpha \Sigma_{\text{expert}})$ with α adapted to keep average TV-distance from the on-policy distribution below a threshold. This produces a covering dataset on which behavior cloning succeeds (with 13.1 mm tracking, vs. crashing). DAGger then iteratively refines the policy by querying the expert on the student’s rollout states; we ran 5 iterations with 1,500 episodes per iteration.

D. Distilled Policy Results

The final policy tracks the figure-8 at 4.1 mm RMSE, 11% degradation from the MPC teacher. Inference is implemented as plain C with hand-written matrix multiplies and runs in 2.1 μ s on the same CPU — $41\times$ faster than the C ADMM solver, with no dynamics model, solver, or external dependency in the deployment binary. A summary is given in Table III.

E. Trade-Offs

The distilled policy is faster and dependency-free, but it is approximate (no constraint guarantees) and uninterpretable (a 6K-parameter black box). The MPC remains the right choice when constraint satisfaction must be certified or when the dynamics model is expected to be reasonably accurate. A complementary approach we did not implement is differentiable MPC [7]: rather than replacing the solver with a network, backpropagate through the solver to learn its parameters (Q , R , model). This preserves constraint satisfaction at deployment while still benefiting from data.

VI. DISCUSSION AND LIMITATIONS

Linearization regime. Tracking results are reported in simulation at 1 m/s, where the body remains within $\sim 15^\circ$ of hover. Beyond 3 m/s linearization error becomes the dominant tracking error source and a switch to nonlinear MPC or differential-flatness feedforward (e.g., the INDI inner loop of [8]) would be required.

Hardware deployment. All numbers in this paper are simulation-side. The C ADMM has been written to be portable to STM32F4-class microcontrollers but has not been flashed to a physical Crazyflie. The published TinyMPC [2] runs at 500 Hz on the same MCU class with similar problem size, providing strong evidence that the deployment is feasible.

ReLU-QP analysis. Our negative result on ReLU-QP is empirical and limited to the specific cost-matrix scales of the Crazyflie linear MPC. We have not characterized the boundary precisely, and a careful study of the conditioning-vs-state-dimension trade-off would be a useful follow-up. The published ReLU-QP scaling experiments use Atlas humanoid and quadruped MPC ($n_x \approx 30\text{--}45$), which sit closer to the crossover and converge cleanly.

Distilled policy generalization. The DAGger student was trained at a fixed mass and disturbance level. We did not run domain randomization or sim-to-real, and we expect tracking to degrade under battery-depletion mass changes, propeller damage, or wind disturbance — closing those gaps would require either training-time domain randomization or test-time adaptation, neither of which is in scope here.

VII. RELATED WORK

Embedded MPC solvers. OSQP [1] is the canonical sparse first-order QP solver; HPIPM [9] provides interior-point methods specialized for MPC. TinyMPC [2], the most direct antecedent for this work, demonstrated 500 Hz MPC on a Crazyflie microcontroller using the same Riccati-caching trick.

Quadrotor MPC. Tal and Karaman [8] reach 66 mm RMSE at 13 m/s on a custom 800 g quadrotor using INDI plus differential-flatness snap tracking. The Crazyswarm Mellinger baseline reports ~ 20 mm at moderate speeds. Our 3.7 mm number at 1 m/s is competitive in the slow regime in simulation; on hardware, 3–5 mm is the practical floor due to airframe flex and motor variability.

Policy distillation for MPC. The MPC-as-teacher idea predates DAGger and has been explored extensively in autonomous driving and locomotion; for quadrotors, Loquercio et al. [10] use IL from privileged simulators, and Song and Scaramuzza [11] explicitly train policies in the loop with MPC on quadrotors. Our contribution here is not the technique but the latency number (2.1 μ s) and the explicit trade-off table.

VIII. CONCLUSION

We have shown that a 400-line, dependency-free C ADMM solver tuned to the Riccati structure of quadrotor MPC tracks a Crazyflie figure-8 at 3.7 mm RMSE with 86 μ s median solve time, $24.9\times$ faster than OSQP. A general-purpose GPU solver (ReLU-QP) is the wrong tool for this size of problem, but its design constraints are well-defined and the failure is informative. Distillation via DAGger compresses the entire stack into a 6K-parameter network at 2.1 μ s deployment latency, with 11% tracking degradation but no dynamics-model dependency. The full pipeline targets the production deployment regime of small-quadrotor autonomy stacks at companies that ship aerial robots.

REFERENCES

- [1] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: an operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [2] A. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, “TinyMPC: Model-predictive control on resource-constrained microcontrollers,” in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024, Best Paper in Automation.
- [3] A. Bishop, J. Brüdigam, and Z. Manchester, “ReLU-QP: A GPU-accelerated quadratic programming solver for model-predictive control,” in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024.
- [4] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [5] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, “DART: Noise injection for robust imitation learning,” in *Conf. on Robot Learning (CoRL)*, 2017.
- [7] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, “Differentiable MPC for end-to-end planning and control,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [8] E. Tal and S. Karaman, “Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness,” *IEEE Trans. Control Systems Technology*, vol. 29, no. 3, pp. 1203–1218, 2021.
- [9] G. Frison and M. Diehl, “HPIPM: A high-performance quadratic programming framework for model predictive control,” in *IFAC World Congress*, 2020.
- [10] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Learning high-speed flight in the wild,” *Science Robotics*, vol. 6, no. 59, 2021.
- [11] Y. Song and D. Scaramuzza, “Policy search for model predictive control with application to agile drone flight,” *IEEE Trans. Robotics*, vol. 38, no. 4, pp. 2114–2130, 2022.